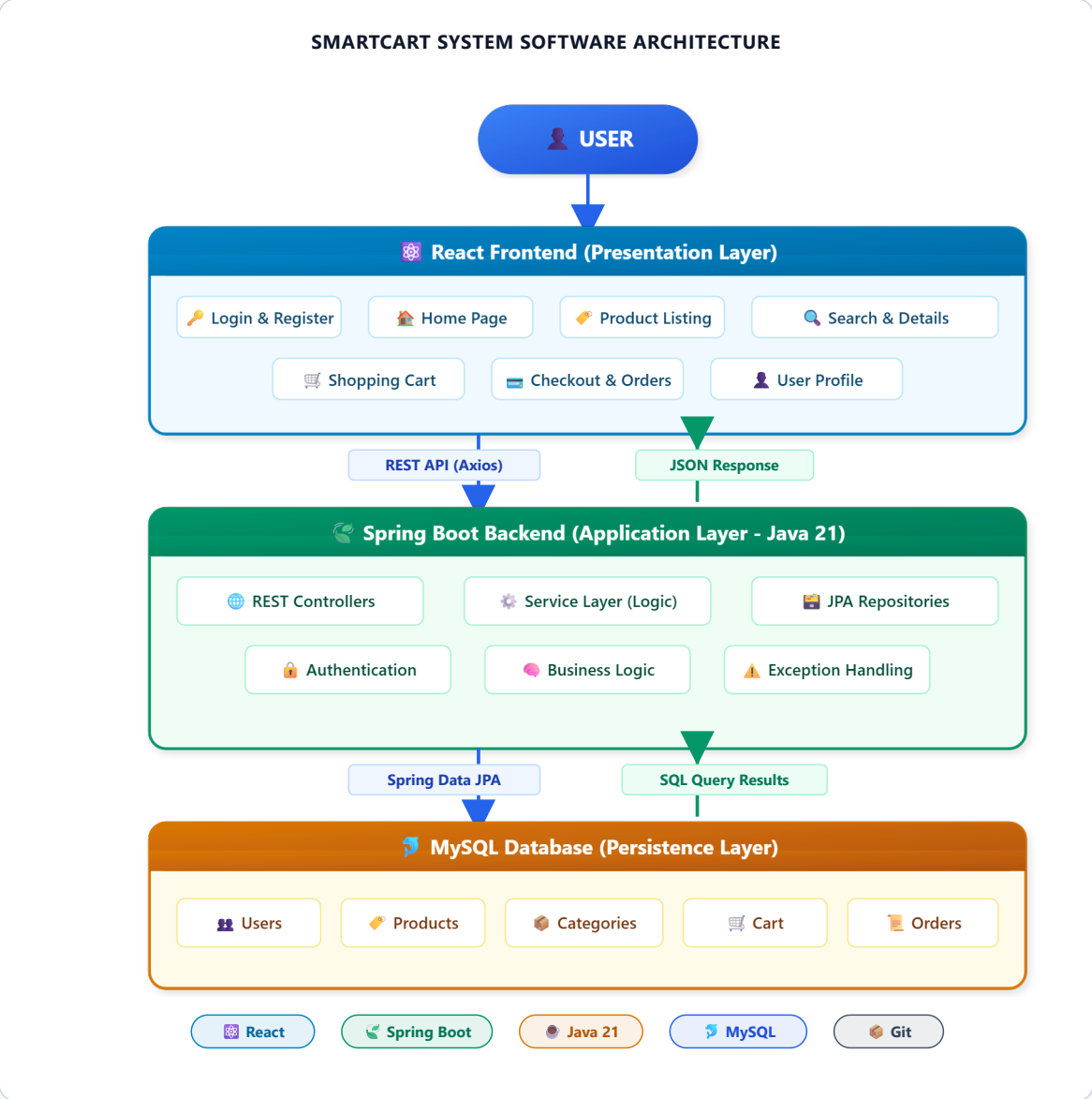
# Project Architecture – SmartCart

Full Stack E-Commerce Shopping Website System Design

**Author:** Technical Software Architect
**Date:** July 2026
**Target:** B.Tech Project Submission

## 1. High-Level Software Architecture Diagram




## 2. Architecture Overview

The **SmartCart** application is engineered following a modern, multi-tiered Client-Server architecture designed for scalability, security, and high performance. The overall system is segmented into four primary operational tiers: the Presentation Layer (Frontend), the API Integration Layer, the Business Logic & Service Layer (Backend), and the Persistence Layer (Database).

At the client side, the user interface is powered by **React.js (v16+)**, creating a dynamic, single-page application (SPA) experience. Users interact with responsive React components for authentication, catalog

browsing, real-time product search, shopping cart management, and order placement. State management is handled cleanly via React Context API, ensuring seamless synchronization across components without external overhead.

Client-server communication is handled asynchronously using the **Axios HTTP library** over standard RESTful endpoints. The backend application is built using **Spring Boot 3.3** on **Java 21**, structured strictly around Controller-Service-Repository design patterns. REST Controllers parse incoming JSON payloads, authenticate user requests, and delegate domain processing to the Service Layer. The service layer executes business validation, transaction handling, and exception management.

Data persistence is managed via **Spring Data JPA (Hibernate)**, which translates Object-Oriented Java domain models into optimized SQL queries executed against a robust **MySQL** relational database. This decoupled separation of concerns guarantees clean maintainability, enterprise-level security, and high performance under modular project expansion.

## 3. Layer-Wise Explanation

### 1. Presentation Layer (React Frontend)

Responsible for rendering the user interface and handling client-side interactions. Built using React components, Context API, and modern CSS3 design tokens. Encapsulates Pages (Home, Products, Categories, Login, Signin, Checkout, Orders, AdminOrders) and UI Components.

### 2. Business Layer (Spring Boot Services)

Contains core business rules, transactional boundaries, validation logic, and authorization workflows. Spring Service classes process input data, compute order totals, validate user sessions, and apply domain rules before triggering persistence.

### 3. Data Access Layer (Spring Data JPA)

Acts as the ORM abstraction layer between Java entities and MySQL database tables. Uses repositories (`UserRepository`, `ProductRepository`, `OrderRepository`) extending `JpaRepository` to provide automated CRUD operations.

### 4. Database Layer (MySQL Server)

Relational database storing persistent enterprise entities. Organizes data into normalized tables: `user`, `product`, `category`, `customer_order`, and `order_item` with relational foreign key constraints.

## 4. Data Flow (Step-by-Step Request Lifecycle)

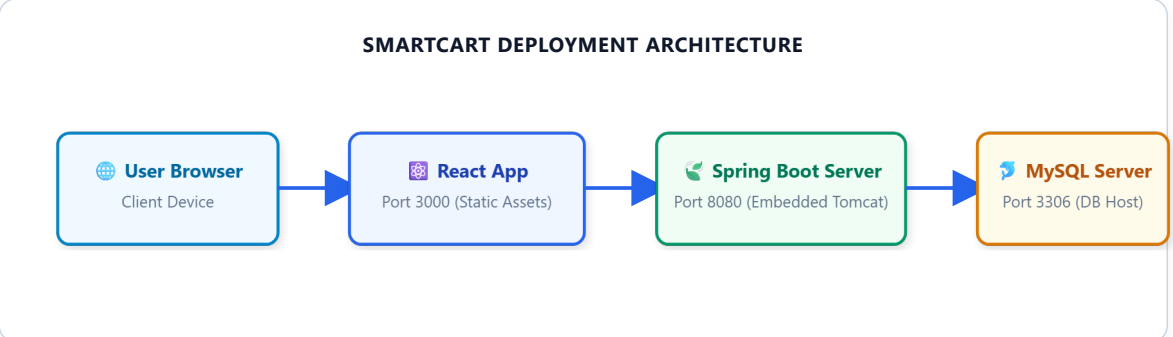
1. **User Action:** The user interacts with the React UI (e.g., clicking "Complete Purchase" on the Checkout page).
2. **Client Request:** React triggers an asynchronous HTTP POST call via `Axios` carrying a structured JSON payload to `/api/orders/create`.
3. **Controller Interception:** Spring Boot's `OrderResource` REST controller intercepts the incoming request, parses the JSON body into an `Order` object, and validates input parameters.
4. **Business Logic Execution:** The backend assigns order timestamps, calculates order status ("PENDING"), and invokes business processing within the service/repository layer.

5. **ORM Translation:** Spring Data JPA converts the Java entity graph into optimized SQL INSERT queries for both customer_order and order_item tables.
6. **Database Persistence:** MySQL executes the SQL queries inside an ACID-compliant transaction and returns generated primary keys.
7. **JSON Response & State Update:** Spring Boot returns an HTTP 200 OK response with the saved Order JSON. React receives the response, clears the shopping cart context, and transitions the UI to the Order Confirmation view.

## 5. Technologies Used

| Technology | Category / Role | Purpose & Description |
|---|---|---|
| **React.js** | Frontend Framework | Renders single-page interface, manages interactive component state & routing. |
| **Spring Boot 3.3** | Backend Framework | Provides enterprise REST API structure, dependency injection, and embedded web server. |
| **Java 21** | Programming Language | Modern LTS core runtime for backend services, OOP domain models, and execution. |
| **Spring Data JPA** | ORM Framework | Automates data mapping, SQL query generation, and repository CRUD operations. |
| **Axios** | HTTP Client | Executes asynchronous REST API requests between React client and Spring Boot. |
| **MySQL** | Relational Database | Stores persistent entity data (Users, Products, Categories, Orders) reliably. |
| **Maven** | Build Tool | Manages backend Java project dependencies, compilation, and executable packaging. |
| **Git** | Version Control | Tracks source code history, branches, and code repository collaboration. |
| **IntelliJ IDEA** | Backend IDE | Primary IDE for Java / Spring Boot development, debugging, and testing. |
| **Visual Studio Code** | Frontend IDE | Lightweight IDE for React, HTML5, CSS3, and JavaScript development. |

## 6. Deployment Architecture



## 7. Component Diagram